Modelación y simulación de tráfico en casetas de peaje

Identificación y generación de variables aleatorias

Dataset: *Mock Traffic Data*, Kaggle (mathfour/mock-traffic-data), archivo `MockTrafficDataForMCNFP.csv`.

Tipo	Variable	Unidad
Discreta	N = número de vehículos que **llegan a la caseta 1** en un intervalo de 1 minuto	vehículos/min
Continua	T = **tiempo de recorrido** de un vehículo entre la caseta 1 y la caseta 2	minutos

Sección	Contenido
0 · Configuración	Librerías y carga de datos
1 · Contexto y preparación	Descripción del dataset y limpieza
Etapa 1–A · Variable discreta	Visualización, candidatas, hipótesis, χ^2, selección
Etapa 1–B · Variable continua	Histogramas, boxplot, normalidad, KS, χ^2, AIC, selección
Etapa 2 · Generación	Generador propio (GCL), transformada inversa → Poisson, Box–Muller → Normal
7 · Conclusiones	Respuestas a las preguntas de la actividad

---

0 · CONFIGURACIÓN

0.1 Librerías y parámetros generales

```
import warnings
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator
from scipy import stats
from scipy.optimize import minimize
from statsmodels.stats.diagnostic import lilliefors
from IPython.display import display
warnings.filterwarnings("ignore", category=FutureWarning)

# ---- Estilo gráfico uniforme ----
C_REAL = "#2a78d6"   # datos reales
C_SIM  = "#eb6834"   # datos simulados / distribución elegida
C_ALT  = ["#1baf7a", "#eda100", "#e87ba4", "#4a3aa7", "#e34948"]  # distribuciones alternativas
plt.rcParams.update({
    "figure.figsize": (10, 4.5), "figure.dpi": 110,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.color": "#e5e4e0", "grid.linewidth": 0.8,
    "axes.edgecolor": "#8a8984", "axes.titleweight": "bold", "axes.titlesize": 12,
    "font.size": 10, "legend.frameon": False,
})
pd.set_option("display.precision", 4)
# Boxplot horizontal: "vert" está obsoleto desde matplotlib 3.10 (se usa "orientation")
HORIZONTAL = ({"orientation": "horizontal"}
              if tuple(int(v) for v in matplotlib.__version__.split(".")[:2]) >= (3, 10) else {"vert": False})

ALPHA = 0.05   # nivel de significancia usado en todas las pruebas
print("Librerías cargadas correctamente.")
```

```
Librerías cargadas correctamente.
```

0.2 Carga de datos

Acepta el `.csv` o el `.zip` descargado de Kaggle. Detecta el separador (`,` o `;`, por si el CSV se guardó desde Excel) y **verifica que estén las columnas esperadas** (`id`, `plate_id`, `time_node_1` … `time_node_6`).

```
import io, zipfile
RUTA_CSV = "MockTrafficDataForMCNFP.csv"        # o "archive.zip", o la ruta completa del archivo
COLUMNAS = ["id", "plate_id"] + [f"time_node_{k}" for k in range(1, 7)]
```

```
def leer_datos(nombre, contenido):
    # Lee un .csv (o el .csv dentro de un .zip) y valida sus columnas
    if nombre.lower().endswith(".zip"):
        with zipfile.ZipFile(io.BytesIO(contenido)) as z:
            csvs = [n for n in z.namelist() if n.lower().endswith(".csv") and not n.startswith("__MACOSX")]
            if not csvs:
                raise ValueError("El .zip no contiene ningún archivo .csv")
            contenido = z.read(csvs[0])
    datos = pd.read_csv(io.BytesIO(contenido), sep=None, engine="python", encoding="utf-8-sig")
    datos.columns = datos.columns.str.strip().str.lower()
    faltan = [c for c in COLUMNAS if c not in datos.columns]
    if faltan:
        raise ValueError(f"El archivo '{nombre}' no es el esperado. Faltan las columnas {faltan}. "
                         f"Columnas encontradas: {list(datos.columns)}")
    return datos

try:
    from google.colab import files                  # solo existe en Google Colab
    nombre, contenido = next(iter(files.upload().items()))   # seleccionar el CSV o el ZIP
except ImportError:
    nombre = RUTA_CSV
    with open(RUTA_CSV, "rb") as f:
        contenido = f.read()

raw = leer_datos(nombre, contenido)
print(f"Archivo '{nombre}' cargado:", raw.shape)
raw.head(10)
```

```
Elegir archivos  MockTrafficD…rMCNFP.csv
MockTrafficDataForMCNFP.csv(text/csv) - 78204 bytes, last modified: 28/9/2026 - 100% done
Saving MockTrafficDataForMCNFP.csv to MockTrafficDataForMCNFP.csv
Archivo 'MockTrafficDataForMCNFP.csv' cargado: (1500, 8)
/usr/local/lib/python3.13/dist-packages/google/colab/_dataframe_summarizer.py:88: UserWarning:
  cast_date_col = pd.to_datetime(column, errors="coerce")
/usr/local/lib/python3.13/dist-packages/google/colab/_dataframe_summarizer.py:88: UserWarning:
  cast_date_col = pd.to_datetime(column, errors="coerce")
```

	id	plate_id	time_node_1	time_node_2	time_node_3	time_node_4	time_node_5	time_node_6
0	0	1N6AD0C	7:00:03	7:11:02	NaN	7:42:11	NaN	8:44:01
1	1	KNDMG4C	7:00:05	NaN	7:47:46	NaN	8:43:54	9:12:38
2	2	5LMJJ3J	7:00:08	7:08:12	NaN	NaN	8:15:32	8:42:42
3	3	WAUDF78	7:00:12	NaN	7:47:53	NaN	8:44:54	9:13:59
4	4	1FAHP3E	7:00:56	NaN	7:52:36	NaN	8:54:49	9:25:31
5	5	1G6DE8E	7:01:34	NaN	7:58:06	NaN	9:05:52	9:38:49
6	6	4T1BF1F	7:02:20	NaN	7:48:40	7:57:56	NaN	8:50:16
7	7	1N4AL2A	7:02:45	7:14:37	NaN	NaN	8:41:25	9:14:56
8	8	KL4CJBS	7:02:47	7:16:57	NaN	7:55:46	8:27:41	8:58:20
9	9	5UXZV4C	7:02:51	NaN	7:56:28	NaN	9:00:14	9:31:30

Próximos pasos: Generar código con raw New interactive sheet

1. Contexto de los datos

El conjunto *Mock Traffic Data* fue creado por Bon Crowder (@mathfour) para una charla de INFORMS (2019) sobre **problemas de flujo de costo mínimo en redes (MCNFP)**. Representa una red vial con **6 casetas (nodos) de registro**: cada vez que un vehículo pasa por una caseta, se registra su placa y la hora.

Columna	Significado
`id`	identificador del registro (viaje)
`plate_id`	placa del vehículo
`time_node_1` … `time_node_6`	hora (hh:mm:ss) en que el vehículo pasó por la caseta *k*; vacío si no pasó por ella

Todos los vehículos **entran por la caseta 1 y salen por la caseta 6**; entre ambas toman distintas rutas (por las casetas 2, 3, 4 y/o 5). Los datos cubren un día de operación, de 7:00 a 19:00 en la caseta de entrada.

> Nota: es un conjunto de datos *simulado* ("mock") con fines didácticos; por eso su comportamiento es más "limpio" que el de un peaje real. Esto se tendrá en cuenta en las conclusiones.

```
info = pd.DataFrame({
    "tipo": raw.dtypes.astype(str),
    "no_nulos": raw.notna().sum(),
    "nulos": raw.isna().sum(),
    "% nulos": (raw.isna().mean() * 100).round(1),
})
info
```

	tipo	no_nulos	nulos	% nulos
id	int64	1500	0	0.0
plate_id	object	1500	0	0.0
time_node_1	object	1500	0	0.0
time_node_2	object	800	700	46.7
time_node_3	object	700	800	53.3
time_node_4	object	870	630	42.0
time_node_5	object	930	570	38.0
time_node_6	object	1500	0	0.0

Próximos pasos: Generar código con `info` | New interactive sheet

1.1 Limpieza y preparación

Pasos aplicados:

1. **Conversión de horas a segundos** desde medianoche (`hh:mm:ss` → número), para poder restar tiempos.
2. **Valores faltantes:** los vacíos en `time_node_2..5` **no son errores**: indican que el vehículo no pasó por esa caseta (tomó otra ruta). No se imputan.
3. **Duplicados:** se revisan `id` (debe ser único) y `plate_id` (una placa puede repetirse si el vehículo hizo más de un viaje).
4. **Consistencia temporal:** para cada viaje, las horas en casetas sucesivas deben ser crecientes.
5. **Rutas:** se identifica el patrón de casetas visitadas por cada vehículo.

```
df = raw.copy()
tcols = [f"time_node_{k}" for k in range(1, 7)]
for c in tcols:
    df[c] = pd.to_timedelta(df[c]).dt.total_seconds()      # segundos desde medianoche

print("ids duplicados:            ", df["id"].duplicated().sum())
print("placas repetidas:          ", df["plate_id"].duplicated().sum(),
      "(vehículos con más de un viaje; se conservan, cada fila es un viaje)")

# Consistencia: tiempos estrictamente crecientes a lo largo del viaje
def es_creciente(fila):
    v = fila[tcols].dropna().values
    return np.all(np.diff(v) > 0)
print("viajes con tiempos no crecientes:", (~df.apply(es_creciente, axis=1)).sum())

# Rutas
df["ruta"] = df[tcols].notna().apply(
    lambda r: "-".join(str(k + 1) for k, v in enumerate(r) if v), axis=1)
rutas = df["ruta"].value_counts().rename_axis("ruta (casetas)").to_frame("vehículos")
rutas["%"] = (rutas["vehículos"] / len(df) * 100).round(1)
rutas
```

```
ids duplicados:             0
placas repetidas:           561 (vehículos con más de un viaje; se conservan, cada fila es un
viajes con tiempos no crecientes: 0
```

ruta (casetas)	vehículos	%
1-3-5-6	500	33.3
1-2-4-6	470	31.3
1-2-4-5-6	200	13.3
1-2-5-6	130	8.7
1-3-4-6	100	6.7
1-3-4-5-6	100	6.7

Próximos pasos: Generar código con rutas | New interactive sheet

Resultado de la limpieza: 1 500 viajes, sin ids duplicados y sin inconsistencias temporales. No fue necesario eliminar registros. Los faltantes corresponden a la estructura de rutas: hay 6 rutas posibles y todas empiezan en la caseta 1 y terminan en la 6.

ETAPA 1-A · VARIABLE DISCRETA

2. Número de vehículos que llegan a la caseta 1

2.1 ¿Qué representa la variable?

N = **número de vehículos que llegan a la caseta de entrada (caseta 1) en un minuto.** Es un **conteo**: toma valores enteros $0, 1, 2, \ldots$ Es la variable de **llegadas** de cualquier modelo de colas de un peaje: determina cuántas cabinas abrir, cuánto crecerá la fila y en qué horas se congestiona la caseta.

```
Comienza a programar o generar con IA.
```

```
# --- Llegadas por hora a la caseta 1 y prueba de homogeneidad de la tasa ---
lleg = df["time_node_1"].values
por_hora = np.histogram(lleg, bins=np.arange(7, 20) * 3600)[0]
chi_h = stats.chisquare(por_hora)        # H0: la tasa de llegadas es igual en todas las horas

fig, ax = plt.subplots(figsize=(10, 3.8))
b = ax.bar([f"{h}h" for h in range(7, 19)], por_hora, color=C_REAL, width=0.6)
ax.axhline(por_hora.mean(), color="#52514e", lw=1.2, ls="--")
ax.text(11.45, por_hora.mean() + 3, f"media = {por_hora.mean():.0f} veh/h", ha="right", color="#52514e")
ax.bar_label(b, padding=2, fontsize=9)
ax.set_title("Llegadas por hora a la caseta 1")
ax.set_ylabel("vehículos"); ax.set_ylim(0, 175)
plt.tight_layout(); plt.show()

print(f"Prueba chi-cuadrado de homogeneidad por hora: X² = {chi_h.statistic:.2f}, gl = 11, p-valor = {chi_h.
print("→ No se rechaza H0 (tasa constante)" if chi_h.pvalue > ALPHA else "→ Se rechaza H0")
```




```
Prueba chi-cuadrado de homogeneidad por hora: X² = 19.14, gl = 11, p-valor = 0.0587
→ No se rechaza H0 (tasa constante)
```

Aunque hay horas con menos llegadas (12h: 96) y con más (14h: 155), la prueba de homogeneidad **no rechaza** (al 5 %) que la tasa de llegada sea la misma durante todo el día (p ≈ 0.059). Por eso se puede trabajar con **una sola tasa λ** para toda la jornada, lo que supone un proceso de llegadas *estacionario*. El p-valor está cerca de 0.05.

```
# --- Construcción de la variable discreta: llegadas por minuto ---
bordes = np.arange(7 * 3600, 19 * 3600 + 1, 60)          # 720 minutos entre 7:00 y 19:00
N = np.histogram(lleg, bins=bordes)[0]
n_obs = len(N)

resumen_N = pd.Series({
    "observaciones (minutos)": n_obs, "total vehículos": N.sum(),
    "media": N.mean(), "varianza": N.var(ddof=1), "desv. estándar": N.std(ddof=1),
    "índice de dispersión (var/media)": N.var(ddof=1) / N.mean(),
    "mínimo": N.min(), "mediana": np.median(N), "máximo": N.max(),
    "asimetría": stats.skew(N), "curtosis (exceso)": stats.kurtosis(N),
})
resumen_N.to_frame("N (veh/min)")
```

	N (veh/min)
observaciones (minutos)	720.0000
total vehículos	1500.0000
media	2.0833
varianza	2.0904
desv. estándar	1.4458
índice de dispersión (var/media)	1.0034
mínimo	0.0000
mediana	2.0000
máximo	8.0000
asimetría	0.7220
curtosis (exceso)	0.3615

```
k_vals = np.arange(0, N.max() + 1)
frec = pd.DataFrame({"k (veh/min)": k_vals, "frecuencia": np.bincount(N)})
frec["frec. relativa"] = frec["frecuencia"] / n_obs
frec["frec. acumulada"] = frec["frec. relativa"].cumsum()
frec
```

	k (veh/min)	frecuencia	frec. relativa	frec. acumulada
0	0	81	0.1125	0.1125
1	1	209	0.2903	0.4028
2	2	180	0.2500	0.6528
3	3	135	0.1875	0.8403
4	4	68	0.0944	0.9347
5	5	33	0.0458	0.9806
6	6	10	0.0139	0.9944
7	7	3	0.0042	0.9986
8	8	1	0.0014	1.0000

Próximos pasos: Generar código con `frec` New interactive sheet

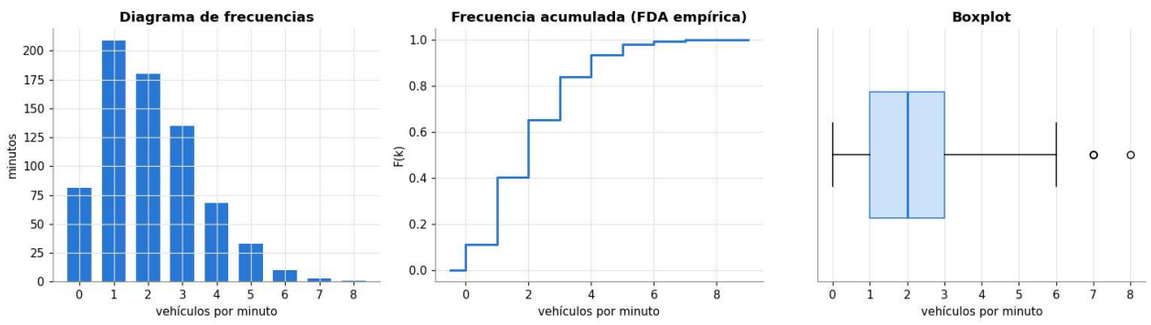
2.2 Visualizaciones

```
fig, axs = plt.subplots(1, 3, figsize=(14, 4))

axs[0].bar(frec["k (veh/min)"], frec["frecuencia"], color=C_REAL, width=0.7)
axs[0].set_title("Diagrama de frecuencias"); axs[0].set_xlabel("vehículos por minuto"); axs[0].set_ylabel("m

axs[1].step(np.r_[-0.5, k_vals, k_vals[-1] + 1], np.r_[0, frec["frec. acumulada"], 1],
            where="post", color=C_REAL, lw=2)
axs[1].set_title("Frecuencia acumulada (FDA empírica)"); axs[1].set_xlabel("vehículos por minuto"); axs[1].s

axs[2].boxplot(N, **HORIZONTAL, widths=0.5, patch_artist=True,
               boxprops=dict(facecolor="#cde2fb", edgecolor=C_REAL), medianprops=dict(color=C_REAL, lw=2))
axs[2].set_title("Boxplot"); axs[2].set_xlabel("vehículos por minuto"); axs[2].set_yticks([])
for a in axs: a.xaxis.set_major_locator(MaxNLocator(integer=True))
plt.tight_layout(); plt.show()
```



Lectura de los gráficos

- La distribución tiene **una sola moda en 1 vehículo/min**, con **cola a la derecha** (asimetría positiva): hay minutos con 5, 6, 7 u 8 vehículos, pero son poco frecuentes. En el 11 % de los minutos no llega ningún vehículo.
- **Media ≈ 2.08 y varianza ≈ 2.09.** Su cociente (índice de dispersión) es prácticamente 1, que es la **propiedad característica de la distribución de Poisson** ($E[N] = Var[N] = \lambda$).
- El boxplot marca como atípicos los minutos con 6 o más vehículos. Son valores **esperables** en un proceso de llegadas aleatorio, no errores, así que se conservan.

2.3 Distribuciones candidatas

Distribución	Justificación en el contexto
Poisson (λ)	Modelo clásico de llegadas independientes y a tasa constante (proceso de Poisson). Encaja con media ≈ variar
Binomial $(n = 60, p)$	Cada segundo del minuto es un "ensayo" en el que llega o no un vehículo. Con n grande y p pequeño se aprox
Binomial negativa (r, p)	Generaliza a Poisson cuando hay **sobredispersión** (var > media), por ejemplo si las llegadas vienen en grupos.
Geométrica (p)	Conteo con moda en 0 y cola decreciente. Se incluye como contraste.

```
lam = N.mean()
p_bin = lam / 60

# Binomial negativa por máxima verosimilitud (parametrización scipy: nbinom(r, p), media = r(1-p)/p)
def nll_nb(theta):
    r, p = np.exp(theta[0]), 1 / (1 + np.exp(-theta[1]))
    return -stats.nbinom.logpmf(N, r, p).sum()
opt = minimize(nll_nb, x0=[np.log(10), 0.0], method="Nelder-Mead", options={"maxiter": 5000, "xatol": 1e-8})
r_nb, p_nb = np.exp(opt.x[0]), 1 / (1 + np.exp(-opt.x[1]))
p_geo = 1 / (1 + lam)

cand_d = {
    "Poisson":           (stats.poisson(lam),        1, f"λ = {lam:.4f}"),
    "Binomial (n=60)":   (stats.binom(60, p_bin),    1, f"n = 60, p = {p_bin:.5f}"),
    "Binomial negativa": (stats.nbinom(r_nb, p_nb),  2, f"r = {r_nb:.1f}, p = {p_nb:.4f}"),
    "Geométrica":        (stats.geom(p_geo, loc=-1), 1, f"p = {p_geo:.4f}"),
}
pd.DataFrame({k: {"parámetros": v[2], "media teórica": v[0].mean(), "varianza teórica": v[0].var()}
              for k, v in cand_d.items()}).T
```

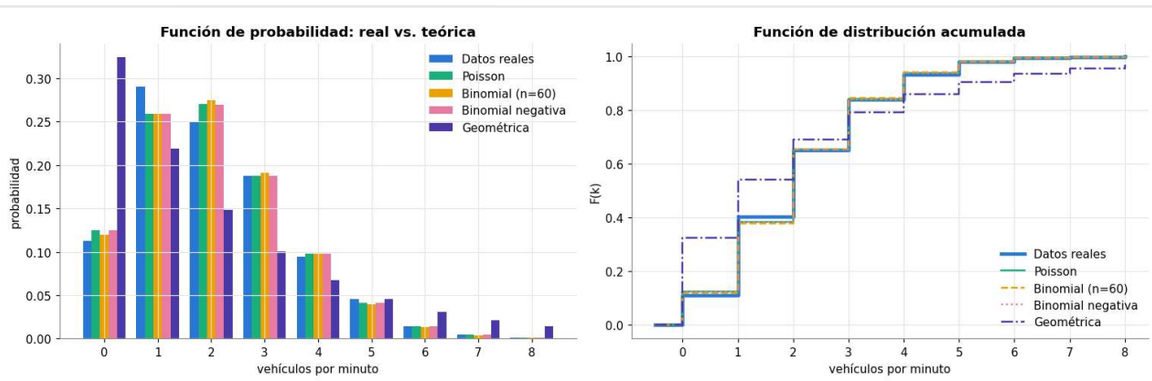
	parámetros	media teórica	varianza teórica
Poisson	λ = 2.0833	2.0833	2.0833
Binomial (n=60)	n = 60, p = 0.03472	2.0833	2.011
Binomial negativa	r = 1055.1, p = 0.9980	2.0833	2.0874
Geométrica	p = 0.3243	2.0833	6.4236

El estimador de máxima verosimilitud de la binomial negativa da un r **muy grande**. Cuando $r \to \infty$ la binomial negativa **converge a la Poisson**: los datos no muestran la sobredispersión que justificaría esta distribución.

2.4 Comparación visual: datos reales vs. distribuciones propuestas

```
fig, axs = plt.subplots(1, 2, figsize=(14, 4.5))
w = 0.16
axs[0].bar(k_vals - 2 * w, frec["frec. relativa"], width=w, color=C_REAL, label="Datos reales")
for i, (nom, (dist, _, _)) in enumerate(cand_d.items()):
    axs[0].bar(k_vals + (i - 1) * w, dist.pmf(k_vals), width=w, color=C_ALT[i], label=nom)
axs[0].set_title("Función de probabilidad: real vs. teórica")
axs[0].set_xlabel("vehículos por minuto"); axs[0].set_ylabel("probabilidad"); axs[0].legend()

axs[1].step(np.r_[-0.5, k_vals], np.r_[0, frec["frec. acumulada"]], where="post", color=C_REAL, lw=3, label=
for i, (nom, (dist, _, _)) in enumerate(cand_d.items()):
    axs[1].step(np.r_[-0.5, k_vals], np.r_[0, dist.cdf(k_vals)], where="post", color=C_ALT[i], lw=1.6,
                ls=["-", "--", ":", "-."][i], label=nom)
axs[1].set_title("Función de distribución acumulada")
axs[1].set_xlabel("vehículos por minuto"); axs[1].set_ylabel("F(k)"); axs[1].legend(loc="lower right")
for a in axs: a.xaxis.set_major_locator(MaxNLocator(integer=True))
plt.tight_layout(); plt.show()
```



Visualmente, **Poisson, binomial y binomial negativa se superponen casi por completo** con los datos reales. La **geométrica queda claramente descartada**: pone la moda en 0 y cae muy rápido, cuando los datos tienen la moda en 1 y más masa en 2–4.

2.5 Hipótesis estadísticas

a) Prueba de dispersión (Poisson vs. sobredispersión)

- H_0: Var(N) = E(N) (los datos son compatibles con Poisson)
- H_1: Var(N) ≠ E(N)
- Estadístico: $D = \frac{(n-1)\,s^2}{\bar{N}} \sim \chi^2_{n-1}$ bajo H_0.

b) Bondad de ajuste chi-cuadrado (una prueba por cada distribución candidata)

- H_0: el número de llegadas por minuto sigue la distribución F_0 (Poisson / binomial / binomial negativa / geométrica).
- H_1: el número de llegadas por minuto **no** sigue la distribución F_0.
- Estadístico: $\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$, con $gl = k - 1 - m$ (k = clases tras agrupar para que $E_i \geq 5$; m = parámetros estimados).
- Nivel de significancia: $\alpha = 0.05$. Se rechaza H_0 si p-valor < 0.05.

Para variables discretas **no se usa Kolmogorov–Smirnov**: la prueba supone una distribución continua y con datos discretos es conservadora. La chi-cuadrado es la adecuada.

```
# a) Prueba de dispersión
D = (n_obs - 1) * N.var(ddof=1) / N.mean()
p_disp = 2 * min(stats.chi2.cdf(D, n_obs - 1), stats.chi2.sf(D, n_obs - 1))
print(f"Índice de dispersión = {N.var(ddof=1)/N.mean():.4f}")
print(f"D = {D:.2f}  ~ χ²({n_obs-1});  p-valor (bilateral) = {p_disp:.4f}")
print("→ No se rechaza H0: no hay sobre ni subdispersión" if p_disp > ALPHA else "→ Se rechaza H0")
```

```
Índice de dispersión = 1.0034
D = 721.44  ~ χ²(719);  p-valor (bilateral) = 0.9348
→ No se rechaza H0: no hay sobre ni subdispersión
```

```
# b) Chi-cuadrado de bondad de ajuste con agrupación de colas (E_i >= 5)
def chi2_discreta(datos, dist, m_param, kmax=None):
    kmax = datos.max() if kmax is None else kmax
    ks = np.arange(0, kmax + 1)
    O = np.bincount(datos, minlength=kmax + 1).astype(float)
    p = dist.pmf(ks); p[-1] = dist.sf(kmax - 1)          # la última clase acumula la cola (k >= kmax)
    p[0] = dist.cdf(0)                                   # la primera clase acumula k <= 0
    E = p * len(datos)
    O, E = list(O), list(E)
    lo, hi = 0, kmax                                     # la clase i representa el valor lo+i
    while E[-1] < 5:                                     # agrupar cola derecha
        e, o = E.pop(), O.pop(); E[-1] += e; O[-1] += o; hi -= 1
    while E[0] < 5:                                      # agrupar cola izquierda
        e, o = E.pop(0), O.pop(0); E[0] += e; O[0] += o; lo += 1
    etiquetas = [str(k) for k in range(lo, hi + 1)]
    etiquetas[0] = f"≤{lo}"; etiquetas[-1] = f"≥{hi}"
    O, E = np.array(O), np.array(E)
    chi = ((O - E) ** 2 / E).sum()
    gl = len(O) - 1 - m_param
    return chi, gl, stats.chi2.sf(chi, gl), pd.DataFrame({"clase": etiquetas, "O_i": O, "E_i": E.round(2),
                                                          "(O-E)²/E": ((O - E) ** 2 / E).round(3)})

res_d, tablas_d = [], {}
for nom, (dist, m, par) in cand_d.items():
    chi, gl, p, tab = chi2_discreta(N, dist, m)
    tablas_d[nom] = tab
    ll = dist.logpmf(N).sum()
    res_d.append({"distribución": nom, "parámetros": par, "χ²": chi, "gl": gl,
                  "χ² crítico (5%)": stats.chi2.ppf(0.95, gl), "p-valor": p,
                  "decisión (α=0.05)": "No se rechaza H0" if p > ALPHA else "Se rechaza H0",
                  "log-verosimilitud": ll, "AIC": 2 * m - 2 * ll})
res_d = pd.DataFrame(res_d).set_index("distribución")
res_d
```

distribución	parámetros	χ^2	gl	χ^2 crítico (5%)	p-valor	decisión (α=0.05)	log-verosimilitud	AIC
Poisson	λ = 2.0833	5.1142	5	11.0705	4.0210e-01	No se rechaza H0	-1241.7677	2485.5355
Binomial (n=60)	n = 60, p = 0.03472	5.6769	5	11.0705	3.3894e-01	No se rechaza H0	-1242.0268	2486.0536
Binomial negativa	r = 1055.1, p = 0.9980	5.1078	4	9.4877	2.7642e-01	No se rechaza H0	-1241.7670	2487.5341
Geométrica	p = 0.3243	275.1284	7	14.0671	1.2281e-55	Se rechaza H0	-1398.7912	2799.5825

Próximos pasos: Generar código con res_d | New interactive sheet

```
print("Tabla de frecuencias observadas vs. esperadas — Poisson")
tablas_d["Poisson"]
```

Tabla de frecuencias observadas vs. esperadas — Poisson

	clase	O_i	E_i	(O-E)²/E
0	≤0	81.0	89.65	0.835
1	1	209.0	186.77	2.645
2	2	180.0	194.55	1.089
3	3	135.0	135.11	0.000
4	4	68.0	70.37	0.080
5	5	33.0	29.32	0.462
6	≥6	14.0	14.23	0.004

2.6 Análisis de resultados

- **Prueba de dispersión:** no se rechaza la igualdad media = varianza. Es la huella de un proceso de Poisson.
- **Poisson:** $\chi^2 \approx 5.1$ con 5 gl, p-valor ≈ 0.40 > 0.05 → **no se rechaza** H_0. Las diferencias entre observado y esperado están repartidas y ninguna clase domina el estadístico.
- **Binomial (n=60):** también se acepta y su ajuste es casi idéntico al de Poisson. Es lo esperable, porque con $n = 60$ y $p \approx 0.035$ la binomial ≈ Poisson.
- **Binomial negativa:** se acepta, pero su r estimado es enorme (en la práctica *es* una Poisson). Además usa **un parámetro más** sin mejorar el ajuste, por eso su AIC no es menor.
- **Geométrica: se rechaza con fuerza** (p-valor ≈ 0).

2.7 Selección y justificación

Distribución seleccionada: Poisson con λ ≈ 2.083 vehículos/minuto (≈ 125 vehículos/hora).

1. **Estadística:** supera la prueba de dispersión y la chi-cuadrado, y tiene el mejor (o empatado) AIC con un solo parámetro.
2. **Gráfica:** la función de probabilidad teórica sigue la forma de los datos (moda en 1, cola a la derecha) y la FDA teórica prácticamente coincide con la empírica.
3. **Contexto:** los vehículos llegan a la caseta **de forma independiente** (un conductor no decide su llegada según los demás), **de uno en uno** y a **tasa constante** durante el día (prueba de homogeneidad). Son exactamente los supuestos de un **proceso de Poisson**, el estándar en teoría de colas para llegadas a un servidor (modelos M/M/c).
4. **Parsimonia:** la binomial y la binomial negativa ajustan igual, pero la binomial obliga a fijar arbitrariamente n (¿segundos?, ¿décimas?) y la binomial negativa añade un parámetro innecesario. Poisson es el modelo más simple con el mismo ajuste.

Consecuencia útil para la simulación: si $N \sim$ Poisson(λ), los **tiempos entre llegadas** son Exponenciales con media $1/\lambda$ ≈ 28.8 s. Esta equivalencia permite usar la variable en una simulación de eventos discretos.

ETAPA 1-B · VARIABLE CONTINUA

3. Tiempo de recorrido entre la caseta 1 y la caseta 2

3.1 ¿Qué representa la variable?

T = **tiempo (en minutos) que dura un vehículo en ir de la caseta 1 a la caseta 2**, es decir, $T = t_{\text{caseta 2}} - t_{\text{caseta 1}}$. Es una variable **continua** (el tiempo puede tomar cualquier valor real positivo).

En el modelo de simulación, T es el **tiempo de servicio o de tránsito** del tramo 1→2: indica cuánto tarda un vehículo en llegar a la siguiente caseta y, por tanto, **cuándo llega a la caseta 2**. Con él se simula la congestión del tramo y la carga de la caseta siguiente.

Se eligió el tramo 1 a 2 porque:

- es el **primer tramo directo** de la red (todos los vehículos que pasan por la caseta 2 vienen directo de la 1): 800 observaciones;
- es un tramo **homogéneo**. El tiempo total 1→6 mezcla rutas de distinta longitud y daría una distribución mezclada (multimodal) que no representa un solo fenómeno.

3.2 Limpieza y preparación

- Se toman solo los vehículos que **pasan por la caseta 2** (800). Los demás (ruta por la caseta 3) no tienen el dato; no es un faltante que haya que imputar.
- Se revisa que no haya tiempos ≤ 0 ni valores imposibles.
- Se identifican atípicos con el criterio de Tukey (1.5·RIC) y se decide si eliminarlos.

```
T = ((df["time_node_2"] - df["time_node_1"]) / 60).dropna().values
print("observaciones:", len(T), "| tiempos <= 0:", (T <= 0).sum(), "| nulos:", np.isnan(T).sum())

q1, q3 = np.percentile(T, [25, 75]); ric = q3 - q1
li, ls = q1 - 1.5 * ric, q3 + 1.5 * ric
atip = T[(T < li) | (T > ls)]
print(f"Límites de Tukey: [{li:.2f}, {ls:.2f}] min  →  {len(atip)} atípicos: {np.sort(atip).round(2)}")

resumen_T = pd.Series({
    "n": len(T), "media": T.mean(), "mediana": np.median(T), "desv. estándar": T.std(ddof=1),
    "coef. variación": T.std(ddof=1) / T.mean(), "mínimo": T.min(), "Q1": q1, "Q3": q3, "máximo": T.max(),
    "asimetría": stats.skew(T), "curtosis (exceso)": stats.kurtosis(T),
})
resumen_T.to_frame("T (min)")
```

```
observaciones: 800 | tiempos <= 0: 0 | nulos: 0
Límites de Tukey: [3.07, 17.02] min  →  5 atípicos: [ 0.3   0.78  2.68 17.08 18.95]
```

	T (min)
n	800.0000
media	10.0175
mediana	10.0333
desv. estándar	2.5969
coef. variación	0.2592
mínimo	0.3000
Q1	8.3000
Q3	11.7875
máximo	18.9500
asimetría	-0.0454
curtosis (exceso)	0.0949

Hay 5 atípicos leves (3 del lado izquierdo y 2 del derecho). El más extremo es un recorrido de 0.3 min (18 s). Con 800 datos normales se esperan unos 5–6 valores fuera de las bandas de Tukey, así que **son compatibles con la variabilidad natural** y **se conservan**: quitarlos reduciría artificialmente la varianza del modelo.

3.3 Visualizaciones

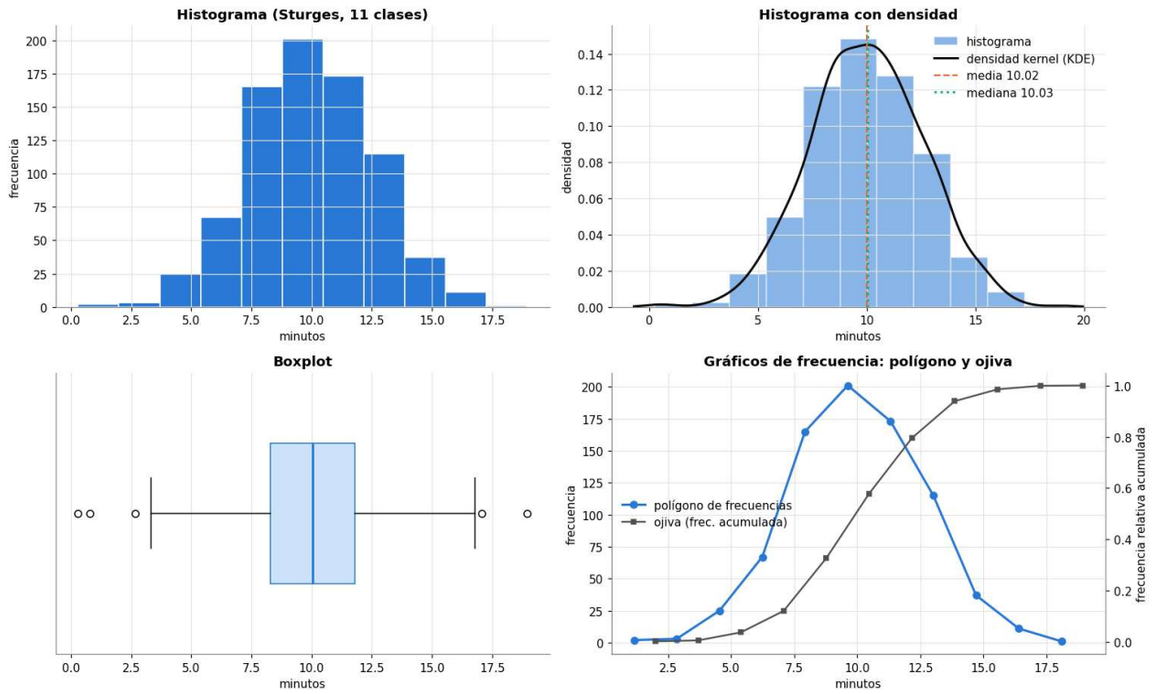
```
fig, axs = plt.subplots(2, 2, figsize=(14, 8.5))
nb = int(np.ceil(np.log2(len(T)) + 1))           # regla de Sturges
bins_T = np.histogram_bin_edges(T, bins=nb)

# Histograma de frecuencias
axs[0, 0].hist(T, bins=bins_T, color=C_REAL, edgecolor="white")
axs[0, 0].set_title(f"Histograma (Sturges, {nb} clases)"); axs[0, 0].set_xlabel("minutos"); axs[0, 0].set_ylab

# Histograma con densidad (KDE)
axs[0, 1].hist(T, bins=bins_T, density=True, color=C_REAL, alpha=0.55, edgecolor="white", label="histograma")
xs = np.linspace(T.min() - 1, T.max() + 1, 400)
axs[0, 1].plot(xs, stats.gaussian_kde(T)(xs), color="#0b0b0b", lw=2, label="densidad kernel (KDE)")
axs[0, 1].axvline(T.mean(), color=C_SIM, ls="--", lw=1.5, label=f"media {T.mean():.2f}")
axs[0, 1].axvline(np.median(T), color=C_ALT[0], ls=":", lw=2, label=f"mediana {np.median(T):.2f}")
axs[0, 1].set_title("Histograma con densidad"); axs[0, 1].set_xlabel("minutos"); axs[0, 1].set_ylabel("densida
axs[0, 1].legend()

# Boxplot
axs[1, 0].boxplot(T, **HORIZONTAL, widths=0.5, patch_artist=True,
                  boxprops=dict(facecolor="#cde2fb", edgecolor=C_REAL), medianprops=dict(color=C_REAL, lw=2))
axs[1, 0].set_title("Boxplot"); axs[1, 0].set_xlabel("minutos"); axs[1, 0].set_yticks([])

# Polígono de frecuencias y ojiva
cnt, _ = np.histogram(T, bins=bins_T)
marcas = (bins_T[:-1] + bins_T[1:]) / 2
ax2 = axs[1, 1]
ax2.plot(marcas, cnt, marker="o", color=C_REAL, lw=2, label="polígono de frecuencias")
ax2.set_ylabel("frecuencia"); ax2.set_xlabel("minutos")
ax3 = ax2.twinx(); ax3.spines["right"].set_visible(True)
ax3.plot(bins_T[1:], np.cumsum(cnt) / len(T), marker="s", ms=4, color="#52514e", lw=1.5, label="ojiva (frec. a
ax3.set_ylabel("frecuencia relativa acumulada"); ax3.grid(False)
ax2.set_title("Gráficos de frecuencia: polígono y ojiva")
h1, l1 = ax2.get_legend_handles_labels(); h2, l2 = ax3.get_legend_handles_labels(); ax2.legend(h1 + h2, l1 + l
plt.tight_layout(); plt.show()
```




```
# Tabla de frecuencias agrupada
tf = pd.DataFrame({"clase (min)": [f"[{a:.2f}, {b:.2f})" for a, b in zip(bins_T[:-1], bins_T[1:])],
                   "marca": marcas.round(2), "frecuencia": cnt})
tf["frec. relativa"] = (tf["frecuencia"] / len(T)).round(4)
tf["frec. acumulada"] = tf["frec. relativa"].cumsum().round(4)
tf
```

	clase (min)	marca	frecuencia	frec. relativa	frec. acumulada
0	[0.30, 2.00)	1.15	2	0.0025	0.0025
1	[2.00, 3.69)	2.84	3	0.0038	0.0063
2	[3.69, 5.39)	4.54	25	0.0312	0.0375
3	[5.39, 7.08)	6.23	67	0.0838	0.1213
4	[7.08, 8.78)	7.93	165	0.2062	0.3275
5	[8.78, 10.47)	9.62	201	0.2512	0.5787
6	[10.47, 12.17)	11.32	173	0.2162	0.7949
7	[12.17, 13.86)	13.02	115	0.1438	0.9387
8	[13.86, 15.56)	14.71	37	0.0462	0.9849
9	[15.56, 17.25)	16.41	11	0.0138	0.9987
10	[17.25, 18.95)	18.10	1	0.0012	0.9999

Próximos pasos: Generar código con `tf` New interactive sheet

Lectura de los gráficos

- El histograma es **unimodal y simétrico**, con forma de campana centrada en ≈ 10 min. **Media (10.02) ≈ mediana (10.03)**, asimetría ≈ –0.05 y curtosis en exceso ≈ 0.1: los tres indicadores apuntan a una forma **normal**.
- El boxplot es simétrico: la mediana está en el centro de la caja y los bigotes tienen longitud similar.
- La ojiva tiene la forma de "S" simétrica típica de una FDA normal.

3.4 Distribuciones candidatas

Distribución	Justificación en e
Normal (μ, σ)	El tiempo de recorrido es la suma de muchos retrasos pequeños e independientes (semáforos, velocidad, adelar
Lognormal (σ, escala)	Clásica para tiempos de viaje en vías congestionadas: positiva y con cola derecha.
Gamma (α, β)	Modelo general de tiempos de servicio positivos. Con α grande se parece a la normal.

Distribución	Justificación en e
Weibull (k, λ)	Muy usada para duraciones. Con $k \approx 3.6$ es casi simétrica.
Exponencial (β)	Supuesto clásico de tiempo de servicio en M/M/1. Se incluye como contraste.

Parámetros estimados por **máxima verosimilitud** (fijando localización 0 en las distribuciones de soporte positivo).

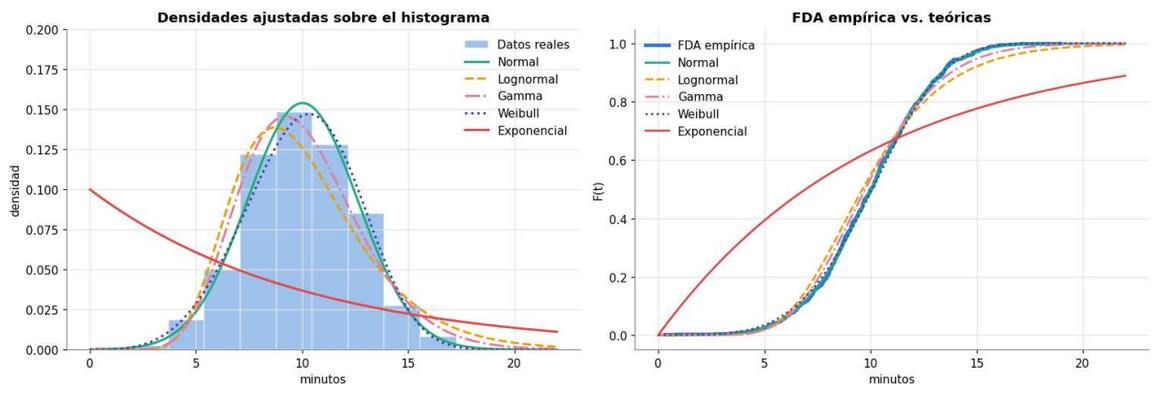
```
mu_T, sd_T = stats.norm.fit(T)
cand_c = {
    "Normal":      (stats.norm,        stats.norm.fit(T),               2),
    "Lognormal":   (stats.lognorm,     stats.lognorm.fit(T, floc=0),    2),
    "Gamma":       (stats.gamma,       stats.gamma.fit(T, floc=0),      2),
    "Weibull":     (stats.weibull_min, stats.weibull_min.fit(T, floc=0), 2),
    "Exponencial": (stats.expon,       stats.expon.fit(T, floc=0),      1),
}
def fmt_par(nom, p):
    if nom == "Normal":      return f"μ = {p[0]:.3f}, σ = {p[1]:.3f}"
    if nom == "Lognormal":   return f"σ_log = {p[0]:.4f}, e^μ = {p[2]:.3f}"
    if nom == "Gamma":       return f"α = {p[0]:.3f}, β = {p[2]:.4f}"
    if nom == "Weibull":     return f"k = {p[0]:.3f}, λ = {p[2]:.3f}"
    return f"media = {p[1]:.3f}"
pd.DataFrame({n: {"parámetros": fmt_par(n, p), "media teórica": d(*p).mean(), "desv. teórica": d(*p).std()}
              for n, (d, p, _) in cand_c.items()}).T
```

	parámetros	media teórica	desv. teórica
Normal	μ = 10.018, σ = 2.595	10.0175	2.5953
Lognormal	σ_log = 0.3148, e^μ = 9.617	10.1061	3.2622
Gamma	α = 12.433, β = 0.8057	10.0175	2.841
Weibull	k = 4.258, λ = 10.991	9.9978	2.6506
Exponencial	media = 10.018	10.0175	10.0175

3.5 Comparación visual: datos reales vs. distribuciones propuestas

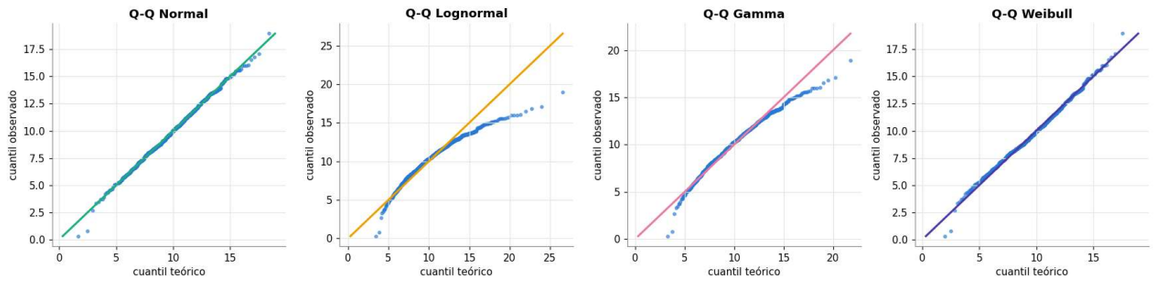
```
fig, axs = plt.subplots(1, 2, figsize=(14, 4.8))
axs[0].hist(T, bins=bins_T, density=True, color=C_REAL, alpha=0.45, edgecolor="white", label="Datos reales")
xs = np.linspace(0, 22, 500)
estilos = ["-", "--", "-.", ":", "-"]
for i, (nom, (d, p, _)) in enumerate(cand_c.items()):
    axs[0].plot(xs, d.pdf(xs, *p), color=C_ALT[i], lw=2, ls=estilos[i], label=nom)
axs[0].set_title("Densidades ajustadas sobre el histograma"); axs[0].set_xlabel("minutos"); axs[0].set_ylabe
axs[0].legend(); axs[0].set_ylim(0, 0.2)

Ts = np.sort(T); Fe = np.arange(1, len(Ts) + 1) / len(Ts)
axs[1].step(Ts, Fe, where="post", color=C_REAL, lw=3, label="FDA empírica")
for i, (nom, (d, p, _)) in enumerate(cand_c.items()):
    axs[1].plot(xs, d.cdf(xs, *p), color=C_ALT[i], lw=1.8, ls=estilos[i], label=nom)
axs[1].set_title("FDA empírica vs. teóricas"); axs[1].set_xlabel("minutos"); axs[1].set_ylabel("F(t)"); axs[
plt.tight_layout(); plt.show()
```




```
# Gráficos Q-Q para las cuatro candidatas principales
fig, axs = plt.subplots(1, 4, figsize=(16, 4))
pp = (np.arange(1, len(Ts) + 1) - 0.5) / len(Ts)
for ax, (nom, (d, p, _)), col in zip(axs, list(cand_c.items())[:4], C_ALT):
    qt = d.ppf(pp, *p)
    ax.scatter(qt, Ts, s=8, color=C_REAL, alpha=0.6)
    lim = [min(qt.min(), Ts.min()), max(qt.max(), Ts.max())]
    ax.plot(lim, lim, color=col, lw=2)
```

```
    ax.set_title(f"Q-Q {nom}"); ax.set_xlabel("cuantil teórico"); ax.set_ylabel("cuantil observado")
plt.tight_layout(); plt.show()
```



En los gráficos Q-Q, la **Normal** tiene los puntos sobre la diagonal en todo el rango. La **Weibull** se desvía poco en las colas. La **Gamma** y sobre todo la **Lognormal** se curvan: tienen asimetría positiva (cola derecha larga) que **los datos no tienen**. La exponencial está claramente fuera (ver densidades).

3.6 Hipótesis estadísticas

a) Pruebas de normalidad

- H_0: el tiempo de recorrido T sigue una distribución normal.
- H_1: T no sigue una distribución normal.
- Pruebas: **Shapiro–Wilk**, **Anderson–Darling** y **Lilliefors** (KS corregida por estimar μ y σ con los mismos datos).

b) Bondad de ajuste para cada candidata F_0

- $H_0: T \sim F_0$ (con los parámetros estimados).
- H_1: T no sigue F_0.
- Pruebas: **Kolmogorov–Smirnov** y **chi-cuadrado** (20 clases equiprobables, $gl = 20 - 1 - m$).
- Criterio complementario: **AIC / BIC** (menor es mejor).
- Como los parámetros se estiman con los mismos datos, el p-valor de KS es aproximado (optimista); por eso se complementa con χ^2 y AIC.
- $\alpha = 0.05$.

3.7 Pruebas de normalidad

```
sw = stats.shapiro(T)
lf = lilliefors(T, dist="norm")
ad = stats.anderson(T, dist="norm")
ad_crit5 = ad.critical_values[list(ad.significance_level).index(5.0)]

norm_tab = pd.DataFrame([
    ["Shapiro-Wilk",    sw.statistic, sw.pvalue],
    ["Lilliefors (KS)", lf[0],        lf[1]],
], columns=["prueba", "estadístico", "p-valor"]).set_index("prueba")
norm_tab["decisión (α=0.05)"] = np.where(norm_tab["p-valor"] > ALPHA, "No se rechaza normalidad", "Se rechaz
print(f"Anderson-Darling: A² = {ad.statistic:.4f}  vs. valor crítico al 5% = {ad_crit5:.3f}  →",
      "No se rechaza normalidad" if ad.statistic < ad_crit5 else "Se rechaza normalidad")
norm_tab
```

```
Anderson-Darling: A² = 0.2096  vs. valor crítico al 5% = 0.783  → No se rechaza normalidad
```

prueba	estadístico	p-valor	decisión (α=0.05)
Shapiro–Wilk	0.9985	0.7472	No se rechaza normalidad
Lilliefors (KS)	0.0177	0.8035	No se rechaza normalidad

Próximos pasos: Generar código con norm_tab | New interactive sheet

3.8 Pruebas de bondad de ajuste para todas las candidatas

```
k_cl = 20
res_c = []
```

```
for nom, (d, p, m) in cand_c.items():
    ll = d.logpdf(T, *p).sum()
    ks = stats.kstest(T, d.cdf, args=p)
    # chi-cuadrado con 20 clases equiprobables según la distribución teórica
    bordes_c = d.ppf(np.linspace(0, 1, k_cl + 1), *p)
    O = np.histogram(T, bins=bordes_c)[0]
    E = np.full(k_cl, len(T) / k_cl)
    chi = ((O - E) ** 2 / E).sum(); gl = k_cl - 1 - m
    res_c.append({"distribución": nom, "parámetros": fmt_par(nom, p),
                  "KS D": ks.statistic, "KS p-valor": ks.pvalue,
                  "χ²": chi, "gl": gl, "χ² p-valor": stats.chi2.sf(chi, gl),
                  "AIC": 2 * m - 2 * ll, "BIC": m * np.log(len(T)) - 2 * ll})
res_c = pd.DataFrame(res_c).set_index("distribución")
res_c["ΔAIC"] = res_c["AIC"] - res_c["AIC"].min()
res_c["decisión (α=0.05)"] = np.where((res_c["KS p-valor"] > ALPHA) & (res_c["χ² p-valor"] > ALPHA),
                                      "No se rechaza H0", "Se rechaza H0")
res_c.sort_values("AIC")
```

distribución	parámetros	KS D	KS p-valor	χ²	gl	χ² p-valor	AIC	BIC	ΔA
Normal	μ = 10.018, σ = 2.595	0.0178	9.5771e-01	9.45	17	9.2508e-01	3800.2135	3809.5827	0.00
Weibull	k = 4.258, λ = 10.991	0.0264	6.2110e-01	17.55	17	4.1775e-01	3817.0651	3826.4343	16.85
Gamma	α = 12.433, β = 0.8057	0.0489	4.1905e-02	44.00	17	3.4250e-04	3901.1671	3910.5363	100.95
Lognormal	σ_log = 0.3148, e^μ = 9.617	0.0763	1.7093e-04	80.90	17	2.6531e-10	4046.8775	4056.2467	246.66
Exponencial	media = 10.018	0.3950	4.9306e-113	1633.70	18	0.0000e+00	5288.9370	5293.6216	1488.72

3.9 Comparación de resultados

Criterio	Normal	Weibull	Gamma	Lognormal	Exponencial
Pruebas de normalidad (3 pruebas)	✓ ninguna rechaza	—	—	—	—
KS	✓	✓	✗	✗	✗
Chi-cuadrado	✓	✓	✗	✗	✗
AIC	**mínimo**	+17	+101	+247	+1 489
Forma (Q-Q)	recta	colas algo desviadas	curva	curva	fuera

- Las **tres pruebas de normalidad coinciden**: ninguna rechaza H_0 (Shapiro–Wilk p ≈ 0.75, Lilliefors p ≈ 0.80, Anderson–Darling por debajo de su valor crítico).
- La **Normal** tiene el menor estadístico KS, el menor χ^2 y el **menor AIC y BIC**.
- La **Weibull** tampoco se rechaza, pero su AIC es ≈ 17 unidades mayor. Una diferencia > 10 es **evidencia fuerte** a favor de la Normal.
- **Gamma y Lognormal** se rechazan (KS y χ^2): imponen una cola derecha que los datos no tienen.
- La **Exponencial** se descarta por completo: el tiempo de recorrido de un tramo **no** es "sin memoria" y tiene una moda clara lejos de 0.

3.10 Selección de la distribución

Distribución seleccionada: Normal con μ ≈ 10.02 min y σ ≈ 2.60 min.

Justificación conjunta:

1. **Gráfica:** histograma simétrico en forma de campana, media ≈ mediana, Q-Q recto.
2. **Estadística:** ninguna prueba rechaza la normalidad y la Normal es la mejor según KS, χ^2, AIC y BIC.
3. **Contexto:** el tiempo de un tramo es la **suma de muchos componentes aleatorios pequeños** (velocidad de cada tramo de vía, intersecciones, comportamiento del conductor). Por el **Teorema**

Central del Límite esa suma tiende a la normal. Además, los datos no muestran la cola derecha típica de la congestión (que justificaría una lognormal).

Advertencia de modelación (pertinente al contexto): la Normal admite valores negativos y **un tiempo de recorrido no puede ser negativo**. Con μ/σ ≈ 3.86, la probabilidad de un valor negativo es $P(T<0)=\Phi(-3.86)\approx 5.7\times10^{-5}$: unos 6 de cada 100 000 vehículos simulados. Es despreciable, pero **en la simulación se usará la Normal truncada en 0**: cualquier valor ≤ 0 se descarta y se vuelve a generar. La **Weibull** ($k\approx4.26$, $\lambda\approx10.99$) queda como alternativa válida si se prefiere una distribución con soporte positivo por construcción.

```
print(f"P(T < 0) bajo Normal({mu_T:.3f}, {sd_T:.3f}) = {stats.norm.cdf(0, mu_T, sd_T):.2e}")
```

```
P(T < 0) bajo Normal(10.018, 2.595) = 5.67e-05
```

ETAPA 2 · GENERACIÓN DE VARIABLES ALEATORIAS

4. Generador de números pseudoaleatorios propio

Se implementa un **Generador Congruencial Lineal (GCL) mixto**:

$$X_{i+1}=(a\,X_i+c)\bmod m,\qquad U_i=\frac{X_i}{m}$$

con los parámetros de *Numerical Recipes*: $a=1\,664\,525$, $c=1\,013\,904\,223$, $m=2^{32}$.

Estos parámetros cumplen el **teorema de Hull–Dobell** (periodo completo $m=2^{32}$ ≈ 4 300 millones):

1. c y m son primos relativos (c es impar y m es potencia de 2);
2. $a-1$ es divisible por todos los factores primos de m (por 2);
3. $a-1$ es divisible por 4, porque m es múltiplo de 4.

Antes de usar los números hay que **verificar que sean uniformes e independientes**.

```
class GCL:
    # Generador congruencial lineal mixto: X_{i+1} = (a X_i + c) mod m
    def __init__(self, semilla=20260927, a=1664525, c=1013904223, m=2**32):
        self.a, self.c, self.m = a, c, m
        self.x = semilla % m
    def aleatorio(self):
        self.x = (self.a * self.x + self.c) % self.m
        return self.x / self.m
    def uniformes(self, n):
        # Devuelve n números U(0,1). Se excluye el 0 exacto (necesario para log(U) en Box-Muller)
        u = np.empty(n)
        for i in range(n):
            v = self.aleatorio()
            while v == 0.0:
                v = self.aleatorio()
            u[i] = v
        return u

# Verificación de condiciones de Hull-Dobell
from math import gcd
g = GCL()
print("gcd(c, m) = 1 :", gcd(g.c, g.m) == 1)
print("(a-1) divisible por 2 y por 4 :", (g.a - 1) % 2 == 0 and (g.a - 1) % 4 == 0)

gen = GCL(semilla=20260927)
U_prueba = gen.uniformes(10_000)
print("Primeros 8 números:", U_prueba[:8].round(6))
```

```
gcd(c, m) = 1 : True
(a-1) divisible por 2 y por 4 : True
Primeros 8 números: [0.407503 0.643164 0.141706 0.555374 0.580673 0.916952 0.189012 0.919595]
```

4.1 Pruebas de calidad del generador (10 000 números)

Prueba	H_0
Chi-cuadrado de frecuencias (20 clases)	Los U_i son uniformes en (0,1)
Kolmogorov–Smirnov	Los U_i son uniformes en (0,1)

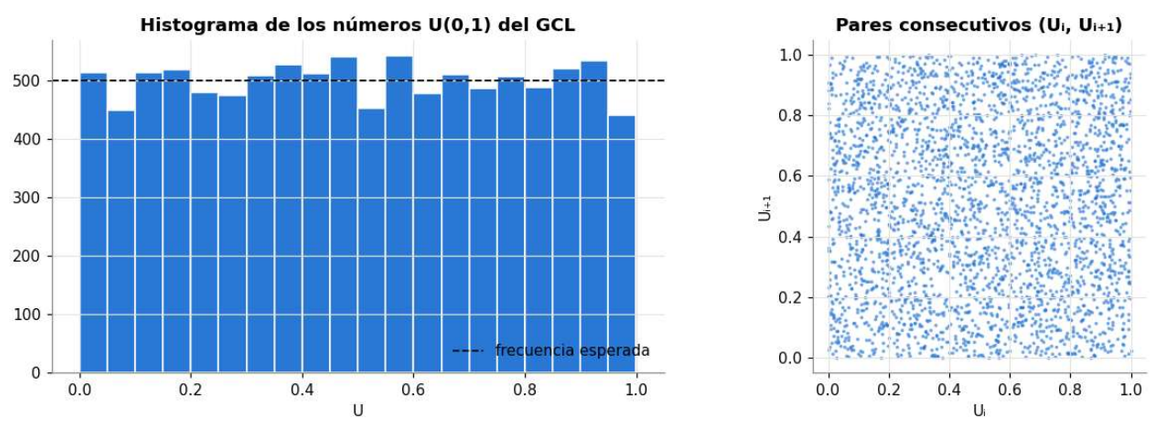
Prueba	H_0
Prueba de medias ($\bar{U}$ vs. 0.5)	$E[U] = 0.5$
Prueba de varianza (s^2 vs. 1/12)	$Var[U] = 1/12$
Corridas arriba/abajo	Los U_i son independientes
Autocorrelación de orden 1	$\rho_1 = 0$ (independencia)

```
def pruebas_uniformidad(u, k=20):
    n = len(u)
    filas = []
    # 1. Chi-cuadrado
    O = np.histogram(u, bins=np.linspace(0, 1, k + 1))[0]; E = n / k
    chi = ((O - E) ** 2 / E).sum(); filas.append(["Chi-cuadrado (20 clases)", chi, stats.chi2.sf(chi, k - 1)])
    # 2. KS
    ks = stats.kstest(u, "uniform"); filas.append(["Kolmogorov-Smirnov", ks.statistic, ks.pvalue])
    # 3. Medias
    z = (u.mean() - 0.5) / np.sqrt(1 / (12 * n)); filas.append(["Prueba de medias (Z)", z, 2 * stats.norm.sf(a
    # 4. Varianza
    x2 = (n - 1) * u.var(ddof=1) / (1 / 12)
    filas.append(["Prueba de varianza (χ²)", x2, 2 * min(stats.chi2.cdf(x2, n - 1), stats.chi2.sf(x2, n - 1))]
    # 5. Corridas arriba y abajo
    s = np.sign(np.diff(u)); s = s[s != 0]
    R = 1 + np.sum(s[1:] != s[:-1])
    mu_R, var_R = (2 * n - 1) / 3, (16 * n - 29) / 90
    zR = (R - mu_R) / np.sqrt(var_R); filas.append(["Corridas arriba/abajo (Z)", zR, 2 * stats.norm.sf(abs(zR)
    # 6. Autocorrelación lag 1
    r1 = np.corrcoef(u[:-1], u[1:])[0, 1]; zr = r1 * np.sqrt(n)
    filas.append(["Autocorrelación lag 1 (Z)", zr, 2 * stats.norm.sf(abs(zr))])
    t = pd.DataFrame(filas, columns=["prueba", "estadístico", "p-valor"]).set_index("prueba")
    t["decisión (α=0.05)"] = np.where(t["p-valor"] > ALPHA, "No se rechaza H0", "Se rechaza H0")
    return t

pruebas_uniformidad(U_prueba)
```

prueba	estadístico	p-valor	decisión (α=0.05)
Chi-cuadrado (20 clases)	35.2080	0.0132	Se rechaza H0
Kolmogorov–Smirnov	0.0066	0.7773	No se rechaza H0
Prueba de medias (Z)	-0.0742	0.9408	No se rechaza H0
Prueba de varianza (χ²)	9891.1187	0.4467	No se rechaza H0
Corridas arriba/abajo (Z)	0.7274	0.4670	No se rechaza H0
Autocorrelación lag 1 (Z)	0.6397	0.5224	No se rechaza H0

```
fig, axs = plt.subplots(1, 2, figsize=(12, 4))
axs[0].hist(U_prueba, bins=20, color=C_REAL, edgecolor="white")
axs[0].axhline(len(U_prueba) / 20, color="#0b0b0b", ls="--", lw=1.2, label="frecuencia esperada")
axs[0].set_title("Histograma de los números U(0,1) del GCL"); axs[0].set_xlabel("U"); axs[0].legend(loc="low
axs[1].scatter(U_prueba[:-1][:3000], U_prueba[1:][:3000], s=2, color=C_REAL, alpha=0.5)
axs[1].set_title("Pares consecutivos (Uᵢ, Uᵢ₊₁)"); axs[1].set_xlabel("Uᵢ"); axs[1].set_ylabel("Uᵢ₊₁")
axs[1].set_aspect("equal")
plt.tight_layout(); plt.show()
```



Con esta semilla, 5 de las 6 pruebas no rechazan H_0. La chi-cuadrado de frecuencias rechaza con p ≈ 0.013. **Eso no prueba por sí solo que el generador sea malo**: con α = 0.05, incluso un generador perfecto rechaza en el 5 % de las muestras, y al aplicar 6 pruebas la probabilidad de que alguna

rechace por azar es ≈ 26 %. Para distinguir un rechazo casual de un defecto real, se repite la batería con **200 semillas distintas**. Si el generador es bueno, cada prueba debe rechazar en ≈ 5 % de los casos.

```
reps = []
for s in range(200):
    t = pruebas_uniformidad(GCL(semilla=101 + 7919 * s).uniformes(10_000))
    reps.append(t["p-valor"])
reps = pd.DataFrame(reps)
tasa = (reps < ALPHA).mean().to_frame("% de rechazos (α=0.05)") * 100
tasa["IC 95 % esperado si el GCL es bueno"] = "2.0 % – 8.0 %"
tasa.round(1)
```

prueba	% de rechazos (α=0.05)	IC 95 % esperado si el GCL es bueno
Chi-cuadrado (20 clases)	6.0	2.0 % – 8.0 %
Kolmogorov–Smirnov	4.5	2.0 % – 8.0 %
Prueba de medias (Z)	4.5	2.0 % – 8.0 %
Prueba de varianza (χ²)	0.5	2.0 % – 8.0 %
Corridas arriba/abajo (Z)	5.0	2.0 % – 8.0 %
Autocorrelación lag 1 (Z)	5.0	2.0 % – 8.0 %

En las 200 repeticiones, las pruebas rechazan en una proporción cercana al 5 % nominal (la prueba de varianza rechaza aún menos, porque su estadístico χ² supone datos normales y con datos uniformes resulta conservadora). **El rechazo de la chi-cuadrado con la primera semilla fue un resultado casual**, no un defecto del generador. El histograma de los U es plano y los pares consecutivos llenan el cuadrado sin patrones visibles. **Los números del GCL son aptos** para transformarlos en variables aleatorias.

---

5. Simulación de la variable discreta: llegadas por minuto ~ Poisson(λ)

5.1 Método: transformada inversa para variables discretas

Para una variable discreta con FDA F, se genera $U \sim U(0,1)$ y se devuelve el **menor k tal que** $F(k) \geq U$. Para Poisson, las probabilidades se calculan de forma recursiva:

$$p_0 = e^{-\lambda}, \qquad p_{k+1} = p_k \cdot \frac{\lambda}{k+1}, \qquad F(k) = \sum_{j=0}^{k} p_j$$

Algoritmo (por cada número aleatorio U):

1. $k = 0$, $p = e^{-\lambda}$, $F = p$.
2. Mientras $U > F$: $k \leftarrow k+1$, $p \leftarrow p\,\lambda/k$, $F \leftarrow F + p$.
3. Devolver k.

```
def poisson_transformada_inversa(lam, U):
    # Genera una muestra Poisson(lam) a partir de números uniformes U, por transformada inversa
    X = np.empty(len(U), dtype=int)
    for i, u in enumerate(U):
        k, p = 0, np.exp(-lam)
        F = p
        while u > F:
            k += 1
            p *= lam / k
            F += p
        X[i] = k
    return X

# Ilustración paso a paso con los 5 primeros números
gen_d = GCL(semilla=12345)
U_d = gen_d.uniformes(100_000)
tabla_inv = pd.DataFrame({"k": np.arange(0, 9), "p(k)": stats.poisson.pmf(np.arange(0, 9), lam),
                          "F(k)": stats.poisson.cdf(np.arange(0, 9), lam)})
print("Tabla de la transformada inversa (λ = %.4f):" % lam)
```

```
display(tabla_inv.round(5))
print("Ejemplo:", [f"U={u:.4f} → k={k}" for u, k in zip(U_d[:5], poisson_transformada_inversa(lam, U_d[:5]))
```

Tabla de la transformada inversa (λ = 2.0833):

	k	p(k)	F(k)
0	0	0.1245	0.1245
1	1	0.2594	0.3839
2	2	0.2702	0.6541
3	3	0.1877	0.8418
4	4	0.0977	0.9395
5	5	0.0407	0.9802
6	6	0.0141	0.9944
7	7	0.0042	0.9986
8	8	0.0011	0.9997

Ejemplo: ['U=0.0204 → k=0', 'U=0.0165 → k=0', 'U=0.5432 → k=2', 'U=0.6349 → k=2', 'U=0.9100 →

```
# Muestra del mismo tamaño que los datos reales (720 minutos = 1 día) y muestra grande (100 000 minutos)
N_sim = poisson_transformada_inversa(lam, U_d[:n_obs])
N_sim_grande = poisson_transformada_inversa(lam, U_d)

comp_d = pd.DataFrame({
    "Real (720 min)": [N.mean(), N.var(ddof=1), N.var(ddof=1) / N.mean(), N.min(), np.median(N), N.max(), N.su
    "Simulado (720 min)": [N_sim.mean(), N_sim.var(ddof=1), N_sim.var(ddof=1) / N_sim.mean(), N_sim.min(), np.
    "Simulado (100 000 min)": [N_sim_grande.mean(), N_sim_grande.var(ddof=1), N_sim_grande.var(ddof=1) / N_sim
                               N_sim_grande.min(), np.median(N_sim_grande), N_sim_grande.max(), np.nan],
    "Teórico Poisson": [lam, lam, 1, 0, stats.poisson.median(lam), np.nan, lam * 720],
}, index=["media", "varianza", "var/media", "mínimo", "mediana", "máximo", "total del día"])
comp_d
```

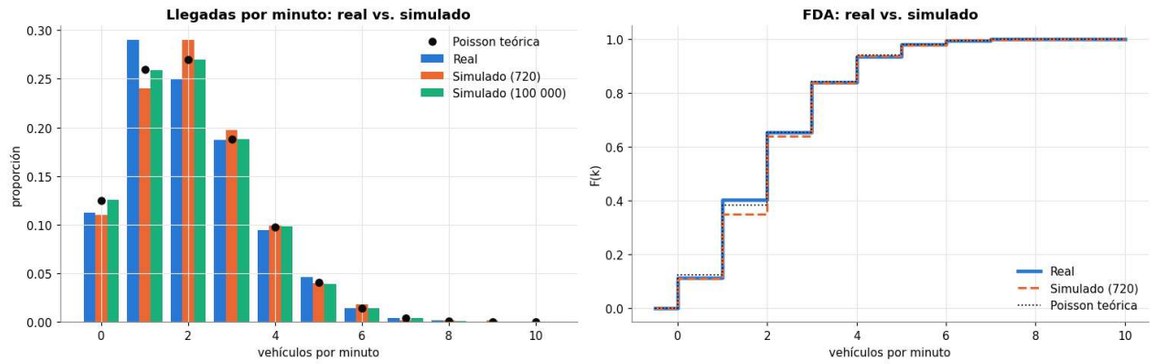
	Real (720 min)	Simulado (720 min)	Simulado (100 000 min)	Teórico Poisson
media	2.0833	2.1556	2.0804	2.0833
varianza	2.0904	2.0592	2.0858	2.0833
var/media	1.0034	0.9553	1.0026	1.0000
mínimo	0.0000	0.0000	0.0000	0.0000
mediana	2.0000	2.0000	2.0000	2.0000
máximo	8.0000	9.0000	11.0000	NaN
total del	1500.0000	1552.0000	NaN	1500.0000

Próximos pasos: Generar código con comp_d | New interactive sheet

```
kk = np.arange(0, 11)
fr_real = np.bincount(N, minlength=11)[:11] / len(N)
fr_sim = np.bincount(N_sim, minlength=11)[:11] / len(N_sim)
fr_simG = np.bincount(N_sim_grande, minlength=11)[:11] / len(N_sim_grande)

fig, axs = plt.subplots(1, 2, figsize=(14, 4.5))
w = 0.27
axs[0].bar(kk - w, fr_real, width=w, color=C_REAL, label="Real")
axs[0].bar(kk, fr_sim, width=w, color=C_SIM, label="Simulado (720)")
axs[0].bar(kk + w, fr_simG, width=w, color=C_ALT[0], label="Simulado (100 000)")
axs[0].plot(kk, stats.poisson.pmf(kk, lam), "o", color="#0b0b0b", ms=6, label="Poisson teórica")
axs[0].set_title("Llegadas por minuto: real vs. simulado"); axs[0].set_xlabel("vehículos por minuto"); axs[0].
axs[0].legend(); axs[0].xaxis.set_major_locator(MaxNLocator(integer=True))

axs[1].step(np.r_[-0.5, kk], np.r_[0, np.cumsum(fr_real)], where="post", color=C_REAL, lw=3, label="Real")
axs[1].step(np.r_[-0.5, kk], np.r_[0, np.cumsum(fr_sim)], where="post", color=C_SIM, lw=2, ls="--", label="Sim
axs[1].step(np.r_[-0.5, kk], np.r_[0, stats.poisson.cdf(kk, lam)], where="post", color="#0b0b0b", lw=1.2, ls="
axs[1].set_title("FDA: real vs. simulado"); axs[1].set_xlabel("vehículos por minuto"); axs[1].set_ylabel("F(k)
plt.tight_layout(); plt.show()
```



5.2 Verificación estadística

1. **Muestra simulada vs. Poisson teórica** (chi-cuadrado de bondad de ajuste). H_0: la muestra simulada sigue Poisson(λ).
2. **Muestra simulada vs. datos reales** (chi-cuadrado de homogeneidad en una tabla de contingencia). H_0: los datos reales y los simulados provienen de la misma distribución.
3. **Prueba de dispersión** sobre la muestra simulada.

```
filas = []
for nombre, muestra in [("Simulado 720", N_sim), ("Simulado 100 000", N_sim_grande)]:
    chi, gl, p, _ = chi2_discreta(muestra, stats.poisson(lam), 0)
    filas.append([f"{nombre} vs. Poisson teórica (bondad de ajuste)", chi, gl, p])

# Homogeneidad real vs simulado (clases 0..5 y >=6)
def agrupa(x, top=6):
    c = np.bincount(np.minimum(x, top), minlength=top + 1); return c
cont = np.vstack([agrupa(N), agrupa(N_sim)])
chi_h2, p_h2, gl_h2, _ = stats.chi2_contingency(cont)
filas.append(["Real vs. Simulado 720 (homogeneidad)", chi_h2, gl_h2, p_h2])

Ds = (len(N_sim) - 1) * N_sim.var(ddof=1) / N_sim.mean()
filas.append(["Dispersión Simulado 720", Ds, len(N_sim) - 1,
              2 * min(stats.chi2.cdf(Ds, len(N_sim) - 1), stats.chi2.sf(Ds, len(N_sim) - 1))])

ver_d = pd.DataFrame(filas, columns=["prueba", "estadístico", "gl", "p-valor"]).set_index("prueba")
ver_d["decisión (α=0.05)"] = np.where(ver_d["p-valor"] > ALPHA, "No se rechaza H0", "Se rechaza H0")
ver_d
```

prueba	estadístico	gl	p-valor	decisión (α=0.05)
Simulado 720 vs. Poisson teórica (bondad de ajuste)	3.9669	6	0.6812	No se rechaza H0
Simulado 100 000 vs. Poisson teórica (bondad de ajuste)	9.0358	10	0.5287	No se rechaza H0
Real vs. Simulado 720 (homogeneidad)	6.2622	6	0.3945	No se rechaza H0
Dispersión Simulado 720	686.8660	719	0.3994	No se rechaza H0

Próximos pasos: Generar código con ver_d | New interactive sheet

5.3 Interpretación

- La muestra simulada reproduce la **media, la varianza y la relación varianza/media ≈ 1** de los datos reales, así como la moda en 1 vehículo/min y la cola hasta 6–8 vehículos.
- **Ninguna prueba rechaza** H_0: la muestra simulada es consistente con Poisson(λ) y **no se distingue estadísticamente** de los datos reales.
- Con 100 000 minutos simulados, las frecuencias convergen a las probabilidades teóricas (ley de los grandes números). Esto confirma que la transformada inversa está bien implementada.

---

6. Simulación de la variable continua: tiempo de recorrido ~ Normal(μ, σ)

6.1 Método: transformación de Box–Muller

La FDA normal no tiene inversa cerrada, así que se usa el **método de Box–Muller**, que transforma **dos** uniformes independientes U_1, U_2 en **dos** normales estándar independientes:

$$Z_1 = \sqrt{-2\ln U_1}\,\cos(2\pi U_2), \qquad Z_2 = \sqrt{-2\ln U_1}\,\sin(2\pi U_2)$$

Luego se escala: $T = \mu + \sigma Z$.

Truncamiento en 0 (por contexto): si algún $T \le 0$, se descarta y se genera otro (método de rechazo). Así el tiempo simulado siempre es positivo.

```
def normal_box_muller(mu, sigma, n, generador, truncar_en_cero=True):
    salida, rechazados = [], 0
    while len(salida) < n:
        falta = n - len(salida)
        m = (falta + 1) // 2
        U1, U2 = generador.uniformes(m), generador.uniformes(m)
        R = np.sqrt(-2 * np.log(U1))
        Z = np.concatenate([R * np.cos(2 * np.pi * U2), R * np.sin(2 * np.pi * U2)])
        X = mu + sigma * Z
        if truncar_en_cero:
            rechazados += (X <= 0).sum()
            X = X[X > 0]
        salida.extend(X[:falta])
    return np.array(salida), rechazados

# Ejemplo: la función toma U1 del primer bloque de m uniformes y U2 del segundo bloque
m_ej = (len(T) + 1) // 2
u_ej = GCL(semilla=777).uniformes(2 * m_ej)
u1, u2 = u_ej[0], u_ej[m_ej]
z1 = np.sqrt(-2 * np.log(u1)) * np.cos(2 * np.pi * u2)
z2 = np.sqrt(-2 * np.log(u1)) * np.sin(2 * np.pi * u2)
print(f"Ejemplo: U1 = {u1:.4f}, U2 = {u2:.4f} → Z1 = {z1:.4f}, Z2 = {z2:.4f} → T = μ + σ·Z1 = {mu_T + sd_T * z

gen_c = GCL(semilla=777)
T_sim, rech1 = normal_box_muller(mu_T, sd_T, len(T), gen_c)
T_sim_grande, rech2 = normal_box_muller(mu_T, sd_T, 100_000, gen_c)
print(f"Primer tiempo simulado: {T_sim[0]:.3f} min (coincide con el ejemplo)")
print(f"Valores <= 0 descartados: {rech1} en la muestra de {len(T)}, {rech2} en la de 100 000")
```

```
Ejemplo: U1 = 0.5372, U2 = 0.3041 → Z1 = -0.3714, Z2 = 1.0511 → T = μ + σ·Z1 = 9.054 min
Primer tiempo simulado: 9.054 min (coincide con el ejemplo)
Valores <= 0 descartados: 0 en la muestra de 800, 10 en la de 100 000
```

```
def resumen(x):
    return [len(x), x.mean(), x.std(ddof=1), np.median(x), x.min(), np.percentile(x, 25), np.percentile(x, 75)
            stats.skew(x), stats.kurtosis(x)]
idx = ["n", "media", "desv. estándar", "mediana", "mínimo", "Q1", "Q3", "máximo", "asimetría", "curtosis (exce
pd.DataFrame({"Real": resumen(T), "Simulado (800)": resumen(T_sim), "Simulado (100 000)": resumen(T_sim_grande
              "Teórico Normal": [np.nan, mu_T, sd_T, mu_T, np.nan, stats.norm.ppf(.25, mu_T, sd_T),
                                 stats.norm.ppf(.75, mu_T, sd_T), np.nan, 0, 0]}, index=idx)
```

	Real	Simulado (800)	Simulado (100 000)	Teórico Normal
n	800.0000	800.0000	100000.0000	NaN
media	10.0175	9.7787	10.0189	10.0175
desv. estándar	2.5969	2.5595	2.5912	2.5953
mediana	10.0333	9.6786	10.0242	10.0175
mínimo	0.3000	1.3132	0.1068	NaN
Q1	8.3000	8.1391	8.2778	8.2670
Q3	11.7875	11.5880	11.7665	11.7680
máximo	18.9500	17.1737	21.3789	NaN
asimetría	-0.0454	-0.0263	-0.0067	0.0000
curtosis (exceso)	0.0949	-0.0267	-0.0142	0.0000

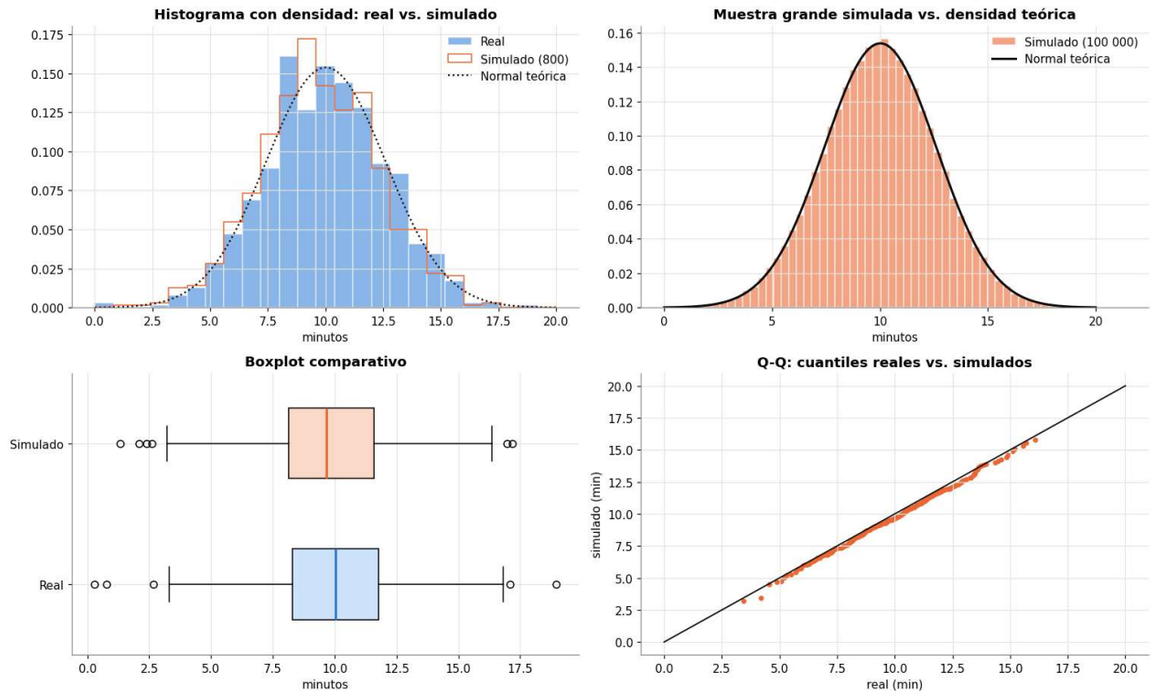
```
fig, axs = plt.subplots(2, 2, figsize=(14, 8.5))
```

```
b = np.linspace(0, 20, 26)
axs[0, 0].hist(T, bins=b, density=True, color=C_REAL, alpha=0.55, edgecolor="white", label="Real")
axs[0, 0].hist(T_sim, bins=b, density=True, histtype="step", color=C_SIM, lw=2.2, label="Simulado (800)")
xs = np.linspace(0, 20, 400)
axs[0, 0].plot(xs, stats.norm.pdf(xs, mu_T, sd_T), color="#0b0b0b", lw=1.5, ls=":", label="Normal teórica")
axs[0, 0].set_title("Histograma con densidad: real vs. simulado"); axs[0, 0].set_xlabel("minutos"); axs[0, 0].

axs[0, 1].hist(T_sim_grande, bins=60, density=True, color=C_SIM, alpha=0.6, edgecolor="white", label="Simulado
axs[0, 1].plot(xs, stats.norm.pdf(xs, mu_T, sd_T), color="#0b0b0b", lw=2, label="Normal teórica")
axs[0, 1].set_title("Muestra grande simulada vs. densidad teórica"); axs[0, 1].set_xlabel("minutos"); axs[0, 1

bp = axs[1, 0].boxplot([T, T_sim], **HORIZONTAL, widths=0.5, patch_artist=True)
axs[1, 0].set_yticks([1, 2], ["Real", "Simulado"])
for patch, col in zip(bp["boxes"], ["#cde2fb", "#fbd9c9"]): patch.set_facecolor(col)
for med, col in zip(bp["medians"], [C_REAL, C_SIM]): med.set_color(col); med.set_linewidth(2)
axs[1, 0].set_title("Boxplot comparativo"); axs[1, 0].set_xlabel("minutos")

qs = np.linspace(0.005, 0.995, 199)
axs[1, 1].scatter(np.quantile(T, qs), np.quantile(T_sim, qs), s=12, color=C_SIM)
axs[1, 1].plot([0, 20], [0, 20], color="#0b0b0b", lw=1.2)
axs[1, 1].set_title("Q-Q: cuantiles reales vs. simulados"); axs[1, 1].set_xlabel("real (min)"); axs[1, 1].set_
plt.tight_layout(); plt.show()
```



6.2 Verificación estadística

Prueba	H_0
Shapiro–Wilk (simulado)	La muestra simulada es normal
KS de una muestra (simulado vs. Normal(μ,σ))	La muestra simulada sigue la Normal seleccionada
KS de dos muestras (real vs. simulado)	Real y simulado provienen de la misma distribución
t de Welch (medias)	$\mu_{real} = \mu_{sim}$
Levene (varianzas)	$\sigma^2_{real} = \sigma^2_{sim}$

```
filas = []
sw_s = stats.shapiro(T_sim);                                filas.append(["Shapiro-Wilk (simulado 800)", sw_s.st
ks1 = stats.kstest(T_sim, stats.norm(mu_T, sd_T).cdf);      filas.append(["KS simulado 800 vs. Normal(μ,σ)", ks
ks1g = stats.kstest(T_sim_grande, stats.norm(mu_T, sd_T).cdf); filas.append(["KS simulado 100 000 vs. Normal(μ
ks2 = stats.ks_2samp(T, T_sim);                             filas.append(["KS dos muestras: real vs. simulado",
tw = stats.ttest_ind(T, T_sim, equal_var=False);            filas.append(["t de Welch (medias)", tw.statistic, t
lv = stats.levene(T, T_sim);                                filas.append(["Levene (varianzas)", lv.statistic, lv
ver_c = pd.DataFrame(filas, columns=["prueba", "estadístico", "p-valor"]).set_index("prueba")
ver_c["decisión (α=0.05)"] = np.where(ver_c["p-valor"] > ALPHA, "No se rechaza H0", "Se rechaza H0")
ver_c
```

prueba	estadístico	p-valor	decisión (α=0.05)
Shapiro–Wilk (simulado 800)	0.9988	0.8615	No se rechaza H0